P2P 네트워크 아키텍처

개요

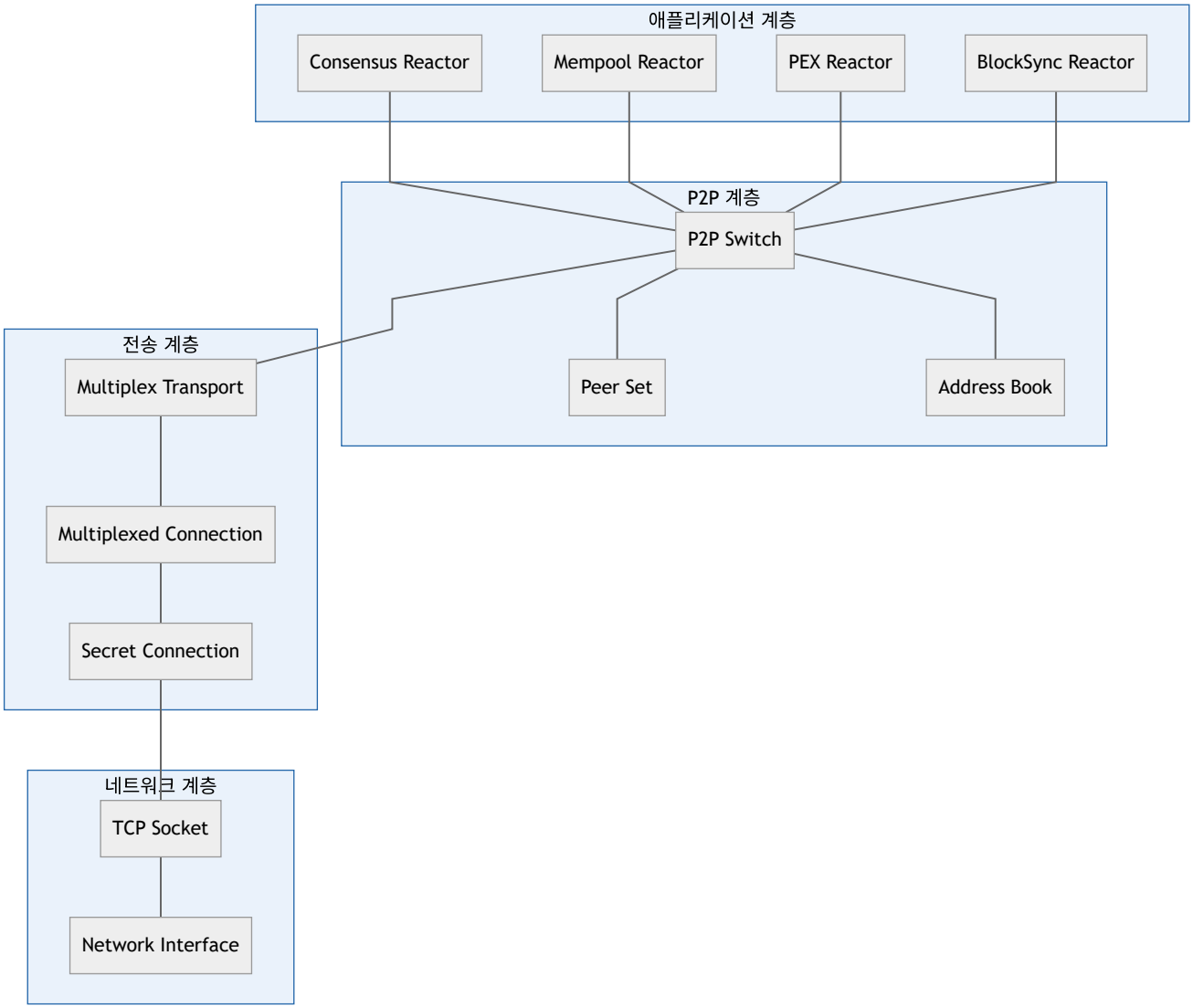
CometBFT의 P2P 네트워킹 시스템은 분산 블록체인 네트워크에서 노드 간 통신을 담당하는 핵심 컴포넌트입니다. 이 시스템은 피어 발견, 연결 관리, 메시지 전파, 보안 통신 등의 기능을 제공하여 안정적이고 확장 가능한 네트워크를 구축합니다.

P2P 네트워킹 시스템의 주요 목표는 다음과 같습니다:

- **분산성**: 중앙 집중식 서버 없이 노드 간 직접 통신
- **확장성**: 수천 개의 노드가 참여할 수 있는 네트워크 지원
- **내결함성**: 일부 노드의 장애에도 네트워크 기능 유지
- **보안성**: 암호화된 통신과 악의적 노드 차단

전체 아키텍처

CometBFT P2P 네트워킹 시스템은 계층적 구조로 설계되어 있으며, 각 계층은 특정한 책임을 가집니다:



주요 컴포넌트

1. P2P Switch

P2P Switch는 CometBFT P2P 네트워킹의 중앙 허브 역할을 합니다.

주요 기능

- **Reactor 관리**: 다양한 프로토콜 Reactor들을 등록하고 관리
- **피어 연결 관리**: 인바운드/아웃바운드 피어 연결 처리
- **메시지 라우팅**: 수신된 메시지를 적절한 Reactor로 전달
- **연결 제한**: 최대 피어 수 제한 및 연결 품질 관리

핵심 인터페이스

```
type Switch struct {
config *config.P2PConfig
reactors  map[string]Reactor
peers *PeerSet
transport transport.Transport
nodeInfo ni.NodeInfo
nodeKey *nodekey.NodeKey
addrBook AddrBook
}
```

2. Transport Layer

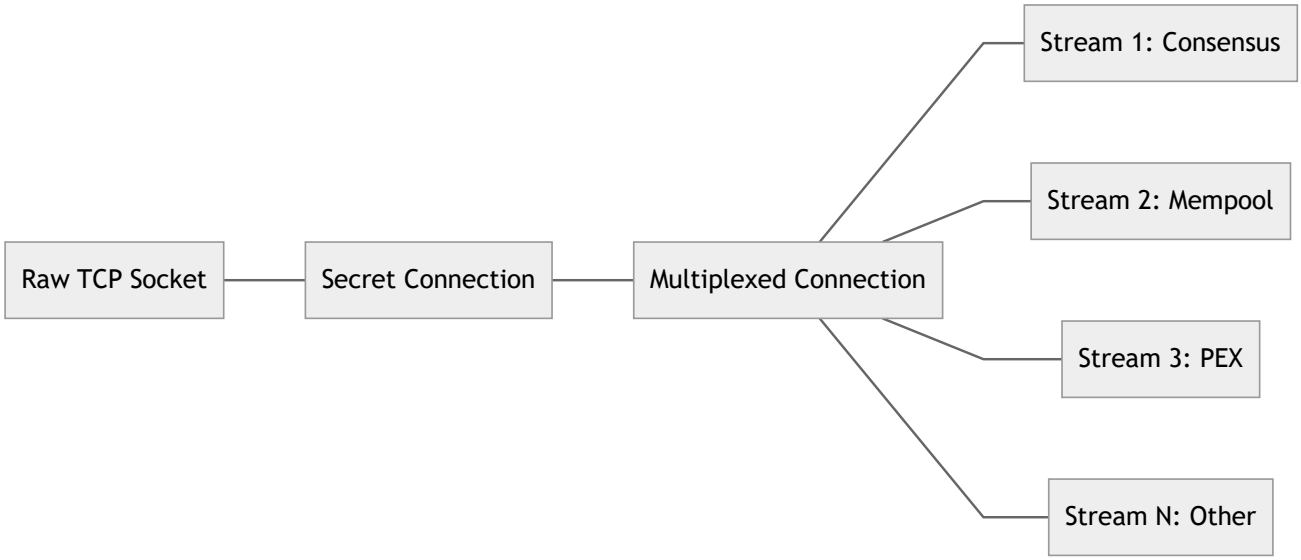
전송 계층은 실제 네트워크 연결을 관리하고 데이터 전송을 담당합니다.

MultiplexTransport

- **TCP 기반**: 안정적인 TCP 연결 사용

- **멀티플렉싱**: 하나의 연결에서 여러 스트림 지원
- **연결 필터링**: 중복 연결 및 악의적 연결 차단
- **암호화**: Secret Connection을 통한 종단간 암호화

연결 계층 구조



3. Peer Management

피어 관리 시스템은 네트워크 참여 노드들의 정보를 관리합니다.

PeerSet

- **활성 피어 추적**: 현재 연결된 피어들의 집합 관리
- **피어 상태 모니터링**: 각 피어의 연결 상태 및 성능 추적
- **동시성 안전**: 멀티스레드 환경에서 안전한 피어 관리

AddrBook (주소록)

- **피어 주소 저장**: 알려진 피어들의 네트워크 주소 관리
- **피어 품질 평가**: Good/Bad 피어 분류 및 평판 관리

- **주소 선택**: 연결할 피어 주소의 지능적 선택

4. Reactor System

Reactor는 특정 프로토콜을 처리하는 컴포넌트입니다.

주요 Reactor 유형

- **Consensus Reactor**: 합의 관련 메시지 처리
- **Mempool Reactor**: 트랜잭션 풀 동기화
- **PEX Reactor**: 피어 교환 프로토콜
- **BlockSync Reactor**: 블록 동기화

Reactor 인터페이스

```
type Reactor interface {
StreamDescriptors() []transport.StreamDescriptor
AddPeer(peer Peer)
RemovePeer(peer Peer, reason any)
Receive(e Envelope)
}
```

네트워크 프로토콜 스택

1. 물리 계층

- **네트워크 인터페이스**: 이더넷, WiFi 등 물리적 연결
- **IP 프로토콜**: IPv4/IPv6 지원

2. 전송 계층

- **TCP**: 신뢰성 있는 연결 지향 프로토콜
- **포트 관리**: 기본 포트 26656 사용

3. 보안 계층

- **Secret Connection**: Ed25519 키 교환 및 ChaCha20-Poly1305 암호화
- **인증**: 노드 신원 확인 및 무결성 검증

4. 멀티플렉싱 계층

- **MConnection**: 단일 TCP 연결에서 여러 논리적 채널 지원
- **스트림 관리**: 각 Reactor별 독립적인 데이터 스트림

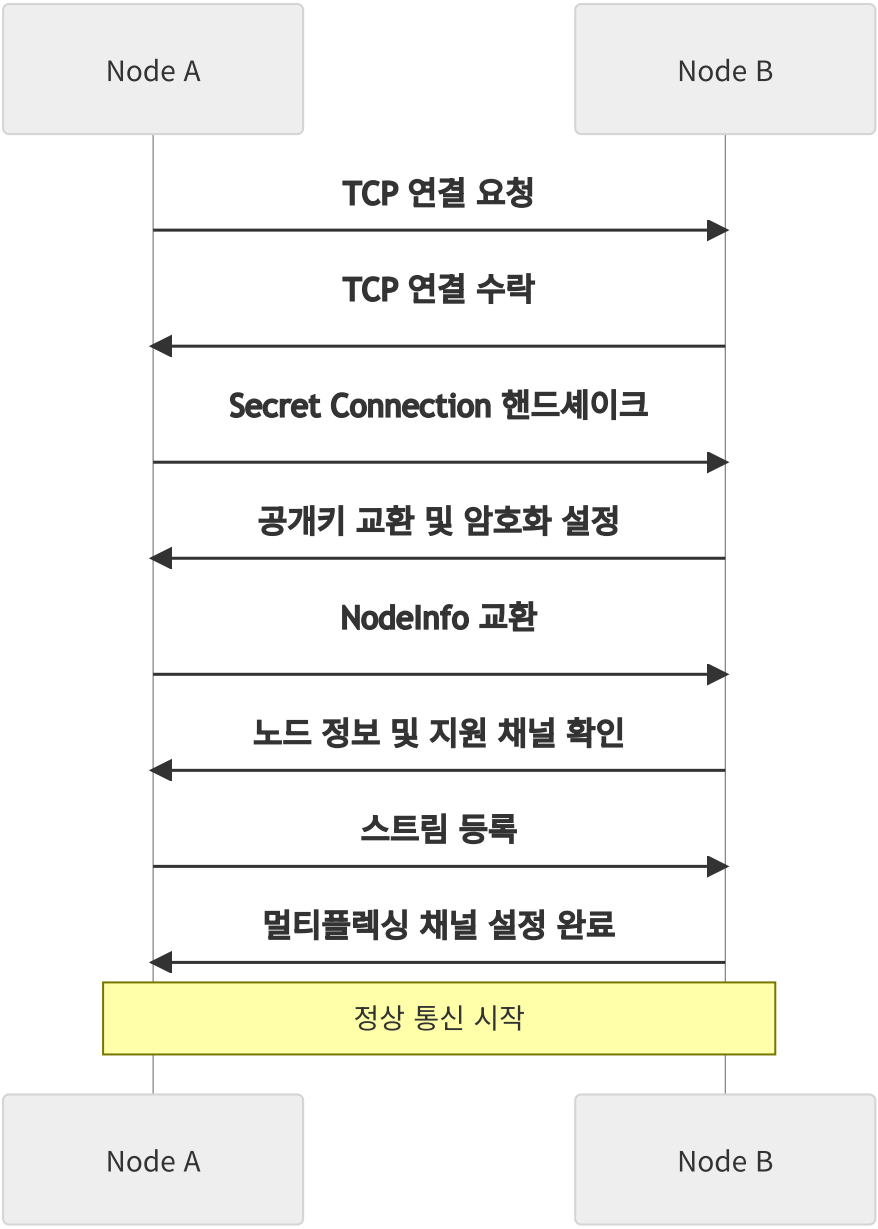
5. 애플리케이션 계층

- **프로토콜 메시지**: Protobuf 기반 구조화된 메시지
- **Reactor 프로토콜**: 각 기능별 특화된 통신 프로토콜

연결 생명주기

---

1. 연결 설정 과정



2. 연결 유지 및 관리

- **Heartbeat**: 주기적인 연결 상태 확인
- **Rate Limiting**: 메시지 전송 속도 제한
- **Quality Monitoring**: 연결 품질 모니터링

3. 연결 종료

- **Graceful Shutdown**: 정상적인 연결 종료 절차
- **Error Handling**: 오류 상황에서의 연결 정리

- **Cleanup**: 리소스 해제 및 상태 정리

네트워크 토폴로지

1. 메시 네트워크 구조

CometBFT는 부분적 메시 네트워크 토폴로지를 사용합니다:

- 각 노드는 제한된 수의 피어와 연결
- 전체 네트워크는 연결된 그래프 형태
- 중앙 집중식 구조 없이 분산된 연결

2. 연결 전략

- **아웃바운드 연결**: 능동적으로 다른 노드에 연결
- **인바운드 연결**: 다른 노드의 연결 요청 수락
- **지속적 피어**: 항상 연결을 유지하려는 특별한 피어
- **시드 노드**: 초기 피어 발견을 위한 부트스트랩 노드

성능 최적화

1. 연결 풀링

- **연결 재사용**: 기존 연결의 효율적 활용
- **연결 제한**: 리소스 사용량 제어
- **부하 분산**: 여러 연결을 통한 트래픽 분산

2. 메시지 최적화

- **배치 처리**: 여러 메시지의 일괄 전송

- **압축**: 대용량 데이터의 압축 전송
- **우선순위**: 중요한 메시지의 우선 처리

3. 네트워크 적응

- **대역폭 감지**: 네트워크 상황에 따른 전송 속도 조절
- **지연 최적화**: 네트워크 지연을 고려한 타이밍 조절
- **오류 복구**: 네트워크 오류 상황에서의 빠른 복구

보안 고려사항

1. 암호화 통신

- **종단간 암호화**: 모든 피어 간 통신 암호화
- **키 관리**: 안전한 키 생성 및 교환
- **무결성 검증**: 메시지 변조 방지

2. 접근 제어

- **피어 인증**: 노드 신원 확인
- **권한 관리**: 기능별 접근 권한 제어
- **블랙리스트**: 악의적 노드 차단

3. DoS 방어

- **Rate Limiting**: 과도한 요청 차단
- **연결 제한**: 동일 IP에서의 다중 연결 제한
- **리소스 보호**: 시스템 리소스 남용 방지

이러한 P2P 네트워크 아키텍처를 통해 CometBFT는 안정적이고 확장 가능한 분산 네트워크를 구축하며, 블록체인 합의 과정에 필요한 신뢰성 있는 통신 기반을 제공합니다.